Minimal SWE Agent 学习问题及解答

基于本次项目对话整理 · 2026-10-03

用途：快速复习你在从零实现第一个 minimal SWE Agent 过程中提出的环境、工程、测试和代码机制问题。重复的“下一步做什么”类问题已合并为阶段路线，避免冗余。

一、项目当前主线

阶段	内容	状态
阶段 0	项目结构、uv、Git	已完成
阶段 1	LLM API（DeepSeek-V3）	已完成
阶段 2	Action / Observation / AgentState	已完成
阶段 3	BaseTool / ToolRegistry	已完成
阶段 4	ReadFile / WriteFile / SearchText	进行中：SearchText 测试
阶段 5	TerminalTool	未开始
阶段 6	Output Parser	未开始
阶段 7	Prompt Builder + ReAct Loop	未开始
阶段 8	自动修复第一个 Bug	未开始
阶段 9	Planner + Git 等扩展	未开始

二、环境与工具

Q1. 做这个简单 SWE Agent 之前，需要准备什么？

答：第一版需要 Python、IDE、uv（或其他包管理）、Git、一个可调用的 LLM API，以及基础的 Python 文件/JSON/异常/子进程知识。暂时不需要 LangChain、Docker、RAG、多 Agent。

Q2. uv 和 Conda 有关系吗？

答：功能有重叠但不是同一个工具。Conda 更偏环境与包管理；uv 是更轻量、更快的 Python 项目与依赖管理工具。这个项目后来决定使用 uv。

Q3. Git 是 GitHub 吗？

答：不是。Git 是本地版本管理工具；GitHub 是托管 Git 仓库的平台。第一阶段只需要本地 Git。

Q4. 可调用的大语言模型是不是要买 API？

答：通常需要 API 服务。聊天订阅和 API 一般是独立的。当前项目使用你提供的平台 API，并先选 DeepSeek-V3 以降低成本。

Q5. Ollama 是什么？现在需要吗？

答：Ollama 用来在本地运行大模型。第一版先不用，避免把精力分散到本地模型部署。

Q6. uv 应该在哪个文件夹初始化？

答：在项目根目录 minimal_swe_agent/ 初始化，也就是与 main.py、pyproject.toml、.env 同级。不要在 swe_agent/、tools/、tests/ 内分别初始化。

Q7. 已经手动建好目录后，为什么用 uv init --bare?

答：因为 --bare 只初始化最基础的项目配置，不额外生成默认 main.py、README 等，适合已经手动搭好目录的项目。

Q8. uv init --bare 后没有 Git，正常吗?

答：正常。uv 不会替你执行 git init。Git 是独立工具，需要在项目根目录单独运行 git init。

三、Git 与 .gitignore

Q1. 我看到 .venv/.gitignore，应该修改这个文件吗?

答：不用。它只负责忽略虚拟环境内部内容。我们需要另外在项目根目录创建 .gitignore。

Q2. .gitignore 里的 .venv/、.env、__pycache__/、*.pyc 哪些是固定格式?

答：都不是必须固定项，它们都是“告诉 Git 忽略什么”的规则。.venv/ 忽略虚拟环境，.env 忽略密钥，__pycache__/ 和 *.pyc 忽略 Python 缓存。

Q3. 哪些文件应该提交到 Git?

答：源代码、tests/、pyproject.toml、uv.lock、根目录 .gitignore 应提交；.venv/、.env、缓存文件不提交。

四、LLM 模块与可扩展性

Q1. 为什么要有 BaseLLM，而不是直接在 Agent 里调用 DeepSeek?

答：为了隔离“Agent 逻辑”和“具体模型接口”。Agent 只调用统一的 generate()；以后换 GPT、Claude、Ollama 时不需要重写 Agent 核心。

Q2. 为什么先单独跑通 API，再写 Agent?

答：先验证“程序能稳定拿到模型回复”，把 API 问题和 Agent 逻辑问题分开，后续调试更清楚。

Q3. 为什么第一版不用 LangChain、AutoGen 等框架?

答：目的是亲手理解 Prompt → LLM → Action → Tool → Observation → State → 下一轮这一条核心链路，避免框架隐藏关键机制。

五、Agent 数据结构

Q1. Action、Observation、AgentStep、AgentState 分别是什么?

答：Action 表示模型决定做什么；Observation 是工具执行结果；AgentStep 是一轮 Action + Observation；AgentState 保存整个任务的 task、plan、history、step_count、finished、final_answer。

Q2. 为什么需要 AgentState，而不是只用一个 history 列表?

答：State 把任务运行状态集中管理，后续做 trajectory、上下文压缩、调试、重放、Planner 都更容易扩展。

六、Tool 与 ToolRegistry

Q1. 为什么要定义 BaseTool?

答：统一规定所有工具必须有 name、description、run()。Agent 不需要关心工具内部实现，只需要按统一接口调用。

Q2. 为什么要 ToolRegistry?

答：为了按名字注册和查找工具，避免 Agent 里出现大量 if/elif。以后新增 git_diff、find_symbol 等工具时，不需要修改 Agent 核心。

Q3. FileToolBase 的意义是什么?

答：把文件类工具共有的 workspace 路径和安全检查集中起来，让 ReadFile、WriteFile、SearchText 自动复用“禁止访问 workspace 外部”的规则。

七、pytest 与测试机制

Q1. 项目里没有 pytest.py 文件，为什么能运行 pytest?

答：pytest 是安装在虚拟环境中的第三方包，同时提供命令行程序。tests/test_xxx.py 才是我们自己写的测试文件。

Q2. 为什么有很多 assert，最后只显示 2 passed?

答：pytest 按测试函数计数，不按 assert 数量计数。一个 test_xxx() 里可以有多个 assert，但仍只算 1 个测试。

Q3. uv run pytest 和 uv run python -m pytest 有什么区别?

答：前者直接启动 pytest 命令；后者明确让当前 uv 环境的 Python 以模块方式运行 pytest。你的环境里后者能更稳定地找到本地 swe_agent 包。

Q4. python -m 里的 m 是什么?

答：m = module。python -m pytest 表示“让当前 Python 解释器找到 pytest 模块并把它作为程序运行”。

Q5. 为什么不加 -m 时会出现 ModuleNotFoundError?

答：你的机器同时有 Conda 和 uv 环境，直接启动 pytest 时模块搜索路径出现差异；python -m pytest 明确使用当前 uv Python，项目根目录更可靠地进入模块搜索路径。

Q6. tmp_path 是什么?

答：pytest 内置 fixture。pytest 看到测试函数参数名 tmp_path 后，会自动创建临时目录并把对应的 Path 对象传进来。

Q7. tmp_path 在哪里给出的?

答：不是你手动赋值，而是 pytest 根据参数名自动注入。可用 uv run python -m pytest -s 配合 print(tmp_path) 查看实际路径。

Q8. tmp_path 创建的临时文件夹测试结束后会删除吗?

答：由 pytest 自动管理，不需要手动删除。某些环境会短期保留最近几次临时目录便于调试，但不会污染项目目录。

Q9. 为什么测试文件读写不用真实 workspace，而用 tmp_path?

答：避免测试污染真实项目数据，并让每个测试拥有独立、可重复的文件环境。

八、Path 与文件操作

Q1. utils_directory = workspace / "utils" 会创建目录吗?

答：不会，只是生成一个 Path 路径对象。真正创建目录的是 utils_directory.mkdir()。

Q2. second_file = utils_directory / "helper.py" 会创建 helper.py 吗?

答：不会，只是表示这个文件路径。真正创建或覆盖文本文件的是 second_file.write_text(...)。

Q3. other.py 为什么不需要提前手动创建?

答：因为 Path.write_text() 在文件不存在时会自动创建文件；如果文件已存在则覆盖内容。

Q4. write_text() 有什么需要特别注意的?

答：它是完整覆盖，不是局部修改。第一版故意保持简单，后续会增加 replace_text / edit_file 一类局部编辑工具。

Q5. 父目录不存在时可以直接 write_text() 吗?

答：不可以。父目录要先 mkdir()，或者像 WriteFileTool 一样先执行 parent.mkdir(parents=True, exist_ok=True)。

九、文件工具与安全

Q1. 为什么文件工具只能操作 workspace?

答：因为 Agent 会自动执行模型决策。限制到 workspace 能避免误读、误写电脑上的其他文件，是第一版最重要的安全边界。

Q2. ReadFileTool 主要做什么?

答：检查 path → 解析为 workspace 内安全路径 → 判断文件存在且是普通文件 → UTF-8 读取 → 返回 Observation。

Q3. WriteFileTool 主要做什么?

答：检查 path/content → 安全解析路径 → 创建需要的父目录 → 创建或覆盖文件 → 返回 Observation，并记录文件之前是否存在。

Q4. SearchTextTool 为什么“没搜到”时 success=True?

答：因为搜索过程本身正常完成，只是匹配数量为 0；“结果为空”和“工具执行失败”是两种不同状态。

Q5. SearchTextTool 为什么能搜索子目录?

答：它用 Path.rglob("*") 递归遍历搜索目录中的文件，再逐行判断 query 是否出现在文本中。

十、项目节奏与上下文

Q1. 隔了很多天没继续做，需要每次都让我回顾前文吗?

答：同一对话中通常不需要，你的核心架构和路线也已经被记录。若换新对话、代码发生较大变化，或你不确定当前版本，再让我先回顾最新代码更稳。

Q2. 当前 minimal SWE Agent 的核心流程最终是什么?

答：用户任务 → Prompt Builder → LLM → Output Parser → Action → ToolRegistry → Tool → Observation → AgentState → 再进入下一轮，直到 Finish。

Q3. 目前下一步是什么?

答：完成 SearchTextTool 测试后，阶段 4 结束；随后进入阶段 5 TerminalTool，让 Agent 可以执行 pytest、python 等终端命令。

十一、快速记忆

- uv 管项目与依赖；Git 管版本；GitHub 托管仓库。
- BaseLLM 隔离模型；BaseTool 统一工具；ToolRegistry 管理工具。
- Action = 要做什么；Observation = 做完得到什么；AgentState = 整个任务当前状态。
- pytest 按 test_函数计数；tmp_path 是自动注入的临时目录 fixture。
- Path 只表示路径；mkdir() 创建目录；write_text() 创建或覆盖文本文件。
- 第一版核心目标：Prompt → LLM → Action → Tool → Observation → State → Loop。